

Company: PhonePe

Job Profile: SDE

Offer Type: Internship + Placement

Internship Stipend: 80k

CTC: 33.5L (19.25L base + 5L joining bonus + 8.5L stocks)

Location: Bangalore/Pune

Process Date:

-PPT+OA : 7th July 2025

-Interviews : 8th July 2025

Placement Process:

1. **Online Assessment** [219 candidates appeared for this round]

4 coding questions were to be solved in 90 minutes.

Problem 1 You are given n people, labeled from 1 to n ($\leq 1e5$). There are m existing friendships represented by two arrays A and B , each of size m , where the i -th friendship is between person $A[i]$ and person $B[i]$. Two people are considered friends if there is a direct or indirect connection between them (i.e., if they belong to the same connected group of friends). Your task is to determine the minimum number of new friendships that must be formed so that every person is connected directly or indirectly—in other words, so that everyone belongs to a single connected group.

Approach: I figured it is a connected components problem, I had 3 options - dsu, dfs and bfs. I went with dfs as it was quicker to implement, the answer was the **number of connected components - 1**. I passed all test cases.

Problem 2 You are given n ($\leq 1e4$) employees, some already have been assigned ladoos and some aren't, you need to assign ladoos to the second type of employees such that the difference between ladoos assigned to adjacent employees doesn't exceed k (≤ 30 i believe) and the maximum number of ladoos any employee has is m (≤ 200). You need to return the number of valid ways to assign ladoos.

Approach: I initially thought this was a greedy/math problem but the ambiguous case of adjacent employees with 0 ladoos led me to think it is a dp problem, also $n*m$ being close to $1e6$ assured me that it was. I applied backtracking and memoized it. My state was $memo[i][prev]$ which is the amount of ways I can assign ladoos to employees from $[i, n-1]$ with the value of the last employee's assignment being $prev$. The tricky part was to decide the lower and upper limit of how many ladoos to assign to the i th employee. I took it from $\max(1, prev-k)$ to $\min(m, prev+k)$. I could solve 11/13 test cases with this approach with a few hidden cases not budging even after optimizations.

Problem 3 You are given an integer n ($\leq 1e3$), an integer k , and an array arr of size n consisting of integers. For each element $arr[i]$, you can find its value using the following operation: convert it to its binary representation and define its value as: Sum of $(bit \times position)$ where position is 1-based from right in the binary representation. You need to find the number of pairs (i, j) such that: $value(arr[i]) + value(arr[j]) == k$.

Approach: Due to the low constraints I didn't think much and applied 2 nested loops. I checked each pair using a lambda function for the condition mentioned by iterating bitwise over the numbers and incremented the response. I passed all test cases.

Problem 4 You are given:

- Four houses from Harry Potter: Gryffindor, Hufflepuff, Ravenclaw, and Slytherin
- Two integers: n: number of students m: maximum number of students allowed in each house
- A list of friendships: pairs of integers (a, b) where student a and student b are friends and must be placed in different houses
- A list of strings, where the i-th string contains the allowed houses for the i-th student (e.g., "[["pref1, pref2"], [pref2]]" you had to parse the string and split them accordingly and store their preferences)

Your task is to determine if it is possible to assign every student to exactly one of their preferred houses such that:

1. No pair of friends is placed in the same house
2. No house has more than m students
3. Each student is assigned to one of their preferred houses

Print "possible" if such an assignment exists, otherwise "impossible"

Approach: The question did not specify the number of friendships and the size of the list of strings, we were supposed to take the input as a stream. Since I didn't practice this type of input parsing, I couldn't solve this question. But since the input was small it could've been solved using backtracking i suppose.

Round 1 Summary: I could solve almost 3/4 questions and it was good enough to get me to the next round. My advice would be to not waste time on a question if it doesn't click, solve others and come back later. I solved 1->3->2 and due to increased confidence after solving 3, I could think of 2 without any pressure.

2. Technical Round - 1 [30 candidates appeared for this round]

This round consisted of 2 coding questions which we were supposed to explain and code on pen paper. The interviewer asked for a brief introduction and then introduced himself before moving to the coding problems

Problem 1 Given an array of n ($\leq 1e5$) integers lying in the range $[-1e9, 1e9]$, return the maximum sum of a subarray that starts and ends with the same number. I could immediately think of hashing the minimum prefix sum of each unique number in that array to that number, when I encounter another instance of that number I could subtract the current prefix sum with the minimum prefix sum hashed to that number to get the max total sum of the subarray. Then I updated the map using the current prefix and continued. Finally I coded the solution for my approach. The interviewer was satisfied with my answer and asked the time complexity which was $O(n)$.

Problem 2 Given a parents array of size n ($\leq 1e6$) which represents a binary tree and q ($\leq 1e6$) queries where queries[i]={node,value}, For each query, the task was to find the maximum XOR between the given value and any of the node values along the path from the root to the specified node, I initially gave a brute force dfs solution which ran in $n*q$, the interviewer asked me to optimize it and since values differed from query to query memoization wouldn't work. I thought for some time and then the interviewer hinted at using a trie data structure for storing nodes, I mentioned it had been a long time since I used it for solving problems and still attempted it, with a few more hints I could reach the solution. I needed to dynamically

create the trie and for each leaf node(in the trie) I could solve the queries that comprised of the tree node it represents. Since the query value is at max $1e6$, its bitwise representation comprises 17-18 bits, making that the height of my trie. The interviewer then asked for the time complexity, which was $O(17*17*n)$.

At the end he asked me whether I had any doubts and I asked about the async nature of PhonePe's systems and how a payment inconsistency is resolved which he explained stating that they constantly poll the NPCI servers for a response after a payment request has been initiated.

3. **Technical Round - 2** [14 candidates appeared for this round]

This round had the same format as technical-1.

Problem 1 Given a matrix of size $m*n$ ($m*n \leq 1e6$), each element had a heat value which I gained if I landed on it, I had to return the minimum heat with which I could reach $\{m-1, n-1\}$ from $\{0,0\}$ under the following constraints:

1. I could move left, right, bottom or top.
2. I could not retrace my path at any point.
3. I couldn't move the same direction more than 3 times.

I mentioned that this seemed like a dijkstra problem to which the interviewer agreed, I considered the state of my heap as: $\{heat, x, y, prev, count\}$ where $\{x, y\}$ are the coordinates of the point that I landed on, heat is the accumulated heat, prev is the last move and count is a counter for move repetition. I maintained a matrix $h[][]$ for storing minimum heat of each position and coded the solution. I mentioned that I could reduce my coordinates from 2 to 1 and compress the matrix h , but I went with this for simplicity. The interviewer agreed with my solution and asked a follow up question. He stated that if there was a minimum limit for moving in one direction before changing direction(say k) how would I adapt my solution. I stated that I could move k distance to the right and bottom of $\{0,0\}$ and push those as the starting points to neglect minimum criteria to which he agreed.

Problem 2 Given 2 arrays cost and time each of size n where $cost[i]$ and $time[i]$ represent the cost of the i th painting and time required to paint it respectively by a paid painter. There was also a magic painter who could paint for free and painted 1 painting per unit time. But the magic painter could only work when the paid painter was busy. I had to return the minimum cost to paint all n paintings. I mentioned that there was a tradeoff between 1)choosing the cheapest painting c for the paid painter and for time $t[c]$ the magic painter could paint $t[c]$ expensive ones, and 2)choosing the most time consuming painting for the paid painter so that magic painter could paint more. I stated that in such a problem I had to consider all paths and hence it is a dp problem to which he agreed. I asked for a minute to think and came up with a state $dp[i][rem]$ which is the minimum cost to paint $[i, n-1]$ paintings where rem is the remaining paintings from $[0, i-1]$. There are essentially two options, use the paid painter and move to $dp[i+1][\max(0, rem-t[i])]$ or don't and move to $dp[i+1][rem+1]$, at the base case check if rem is 0. He agreed and I coded out the solution.

In both of these rounds I made sure to communicate my thought process with the interviewer which is essential for them to understand how you approach a problem.

4. **HM - Round** [11 candidates appeared for this round]

My HM interview was taken by the Engineering Manager at PhonePe. He started off by asking for my introduction and then started questioning me based on my resume. He asked about my internship experience

and what I learnt from it. I answered with an emphasis on the exposure I received to a real world codebase and system, and the design/devops concepts I could implement during it. He asked about the most difficult project, what made it difficult and how would I reapproach it if i could start over. He went on to ask similar behavioural questions regarding my projects and experiences. He then asked me about how I would design instagram stories to which I responded with a rank based algorithm where I'd assign points to each story according to the interaction with that user and give them a rank, I mentioned I'll use a map data structure to hash a pair of {rank,userId} to the array of urls of the photo/video content in the story and then cache stories using redis, or use a cdn. He then asked me about my most recent project and how much help did I take from llms in it to which I honestly replied stating that I used it for the ui of screens which I had a hard time figuring out, and also for debugging code. At the end he asked me if I had questions, I mentioned that in the PPT it was stated that there's a flat hierarchy structure at PhonePe, and I asked what it meant. He explained the hierarchy structure in depth. I also asked about the sports culture at PhonePe which he happily explained. The round went well overall and he wished me luck at the end. The key is to brush up your system design knowledge, both in general and PhonePe in specific, and be honest and expressive with your answers.

Useful Tips:

1)Solve a wide range of problems and emphasize on concepts like binary search, dp, graphs, dsu, tries and a few niche concepts like binary lifting afterwards. Give contests regularly without any hesitation, it's better to underperform there than to do it in your interviews. Read editorials rather than asking llms for solutions. A good way to learn is by solving [striver's sheet](#) and then giving contests on any platform of your choice, then practicing pyqs of companies when placements are close.

2)Know your resume well and research the company you're interviewing for on a basic level at least.

3)Learn system design concepts and real world case studies. Also learn fundamentals like dbms, oops and os; they might not be directly asked all the time but will improve your understanding of development and sys design. For sys design I would recommend [engineering digest](#), for oops [Kunal Kushwaha's playlist](#) and for dbms and os; you could watch [code help's videos](#). You can practice problems on gfg/leetcode.